

CPE 486/586: Deep Learning for Engineering Applications

05 Deep Learning for Sequence Data

Spring 2026

Rahul Bhadani

Outline

1. Motivation
2. Convolutional Neural Network
3. Modern Convolutional Neural Network
4. N-Gram Language Model
5. Recurrent Neural Network
6. Long Short-Term Memory (LSTM)
7. Transformer

Motivation

0	0	0	0	0	0	1	1	1	0	0	0	1	0	1	1	0	1	1	0
1	0	0	1	0	1	1	1	0	1	1	1	0	1	1	0	1	0	0	0
1	0	1	0	0	0	1	0	1	1	0	0	1	0	1	1	0	0	0	1

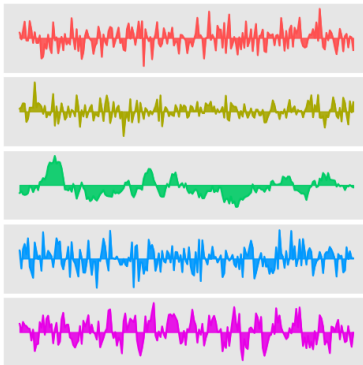
Until Now ...

- 1 Data modality: Non-sequential (tabular with no order for samples or features)
- 2 Process modality: iterative refinement of solution, but not necessarily sequential (non-generative processes – hence no notion of sequential)

Sequential Data



Image/Video



Timeseries

**Efficient modelling
of feature
interactions
underpins**

Texts/Natural
Language/Computer Code

Can we use what we learnt so far on Sequential Data?

Short answer: **May be!** But not quite!

So where is the problem?

Underlying Assumptions

- 1 Core assumptions have been that data samples are iid (independent and identically distributed)
- 2 There are also no temporal or spatial dependencies (order of features in the tabular data didn't matter until now).
- 3 Generally feature length (or number of features are fixed).
- 4 Non-stationarity assumption: data distribution didn't change.

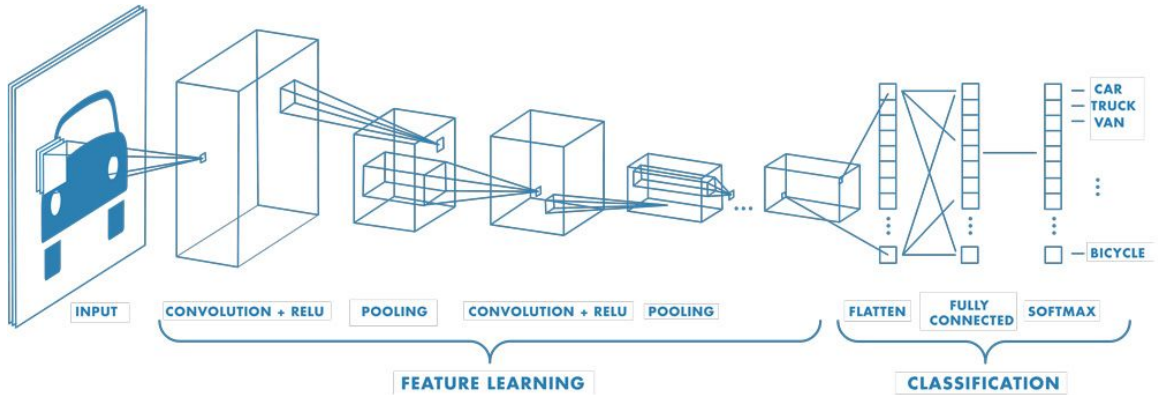
Learning on Sequential Data

- 1 Images: Convolutional Neural Network (CNN)
- 2 Texts/Natural language: Recurrent Neural Network (RNN), Long Short-Term Memory (LSTM) Network, Gated Recurrent Unit (GRU) Network, Transformer
- 3 Videos: CNN + LSTM/GRU/Transformer

Convolutional Neural Network

0	1	0	1	1	0	1	0	1	0	1	0	0	0	0	1	1	1	1	0
0	1	1	0	1	0	0	0	0	0	1	1	1	1	1	1	1	1	0	0
1	0	1	1	0	0	1	0	1	1	1	0	1	0	0	0	0	1	1	0

Convolutional Neural Network

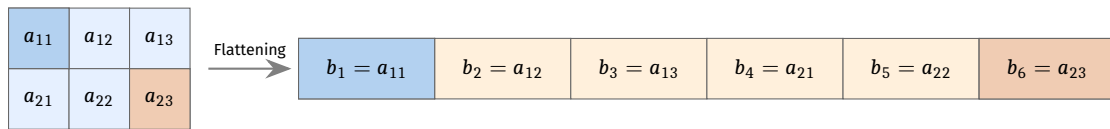


Source:

<https://www.mathworks.com/discovery/convolutional-neural-network.html>

Motivation for CNN

⚡ **(Fully-connected Neural Network, FNN Limitation:** Spatial data must be converted to a **1-D array** (Flattening).



- ✗ **Spatial Loss:** We lose the **pixel-to-pixel relationship** and local geometry.
- ✗ **RGB Complexity:** In addition, working with color images would mean we have at least three channels, R, G, B.
- ✗ **Parameter Explosion:**
 - $200 \times 200 \times 3 = 120,000$ features per image.
 - Just 100 images = **12,000,000** features to process!

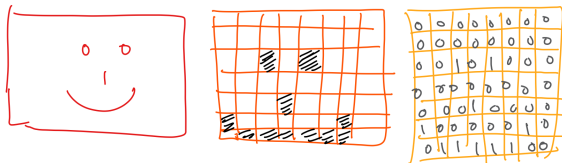
Limitations of FNN for Images

In that sense, FNN suffers from two conditions:

- ⚡ Unable to preserve spatial information
- ⚡ Curse of dimensionality

To preserve the pixel-to-pixel spatial relationship, we include two additional operations in the neural network:

- 1 Convolution
- 2 Pooling



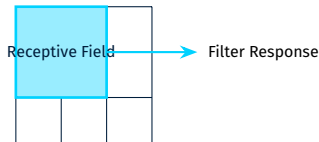
What is a CNN?

Beyond Flattening

A **Convolutional Neural Network (CNN)** is designed for grid-structured data. Unlike FNNs, it treats images as spatial volumes, preserving local relationships.

The Architectural Edge:

- ⚡ **Local Receptive Fields:** Neurons only “see” a small local region, mimicking the human visual cortex.
- ⚡ **Spatial Hierarchy:** Layers learn to recognize edges, then shapes, then objects.



Weight Sharing & Pooling

Weight Sharing

Filters share weights across the entire image.

- ⚡ **Efficiency:** Drastically reduces parameters.
- ⚡ **Feature Detection:** A vertical edge detector works everywhere!

Pooling

Subsampling to reduce spatial dimensions.

- ⚡ **Invariance:** Stable against small translations/shifts.
- ⚡ **Speed:** Lowers computational load & memory.

FNN: 224x224 image $\approx$ 150k+ weights per neuron.

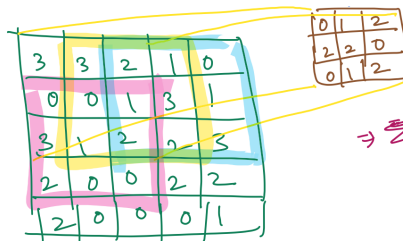
CNN: 3x3 filter = **9 weights** per feature map.

Building blocks: Convolution

Convolution is an operation to reduce the number of pixels through some operation that, in turn, reduces the number of weights to learn.

In convolution, the reduction in the number of features happens by sliding a kernel across an image, performing the dot product.

Kernel: It is an $n \times n$ matrix of some numbers whose dimension is usually smaller than the image size.



5x5 matrix

$$\sum \begin{bmatrix} 3 \times 0 & 3 \times 1 & 2 \times 2 \\ 0 \times 2 & 0 \times 2 & 1 \times 0 \\ 2 \times 0 & 1 \times 1 & 2 \times 2 \end{bmatrix}$$

$$\Rightarrow \sum [0 + 3 + 4 + 0 + 0 + 0 + 0 + 1 + 4] = 12$$

$$\begin{bmatrix} 12 & 12 & 17 \\ 10 & 17 & 19 \\ 9 & 6 & 14 \end{bmatrix}^{3 \times 3}$$

Convolution Using Kernel

3	3	2	1	0
0	0	1	3	1
3	1	2	2	3
2	0	0	2	2
2	0	0	0	1

5x5 Input Matrix

3x3 Kernel

0	1	2
2	2	0
0	1	2

$$\begin{aligned} \sum & \begin{bmatrix} 3 \times 0 & 3 \times 1 & 2 \times 2 \\ 0 \times 2 & 0 \times 2 & 1 \times 0 \\ 3 \times 0 & 1 \times 1 & 2 \times 2 \end{bmatrix} \\ &= 0+3+4+0+0+0+0+1+4 \\ &= 12 \end{aligned}$$

12	12	17
10	17	19
9	6	14

3x3 Output Map

Kernels

A particular kernel usually extracts certain features from an image, or basically, it performs some sort of image processing.

Examples of Kernels

$$\text{Edges: } \begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix}$$

$$\text{Sharpen: } \begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

$$\text{Box Blur: } \frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

Edge-detection Example

Original Image



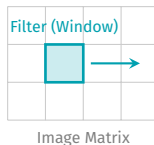
Edge Detection Filtered Image



Filter

The window we use in kernel is called a **filter**. The window of a certain size is slid across the image. We can apply pooling to perform operations such as **minimum**, **median**, **maximum**.

- ⚡ **Images** can be represented as **matrices**, where each element corresponds to a pixel.
- ⚡ A **filter** is just a small matrix that is **convolved** with same-sized sections of the image matrix.



Convolutional Filters

$$\begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 2 & 2 & 1 & 1 & 0 \\ 0 & 2 & 4 & 4 & 2 & 2 & 0 \\ 0 & 1 & 3 & 3 & 1 & 1 & 0 \\ 0 & 1 & 2 & 3 & 1 & 1 & 0 \\ 0 & 0 & 1 & 1 & 0 & 0 & 0 \end{bmatrix} \times \begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix} = \begin{bmatrix} 0 & \dots & \dots \\ \vdots & \ddots & \\ \vdots & & \ddots \end{bmatrix}$$

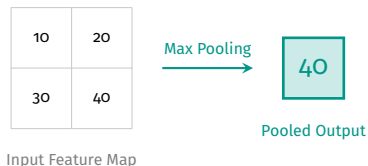
$$\begin{aligned} & (0 \times 0) + (0 \times 1) + (0 \times 0) + \\ & (0 \times 1) + (1 \times -4) + (2 \times 1) + \\ & (0 \times 0) + (2 \times 1) + (4 \times 0) = \mathbf{0} \end{aligned}$$

Convolutional Filters

$$\begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 2 & 2 & 1 & 0 \\ 0 & 2 & 4 & 4 & 2 & 0 \\ 0 & 1 & 3 & 3 & 1 & 0 \\ 0 & 1 & 2 & 3 & 1 & 0 \\ 0 & 0 & 1 & 1 & 0 & 0 \end{bmatrix} \times \begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix} = \begin{bmatrix} 0 & -1 & -1 & 0 \\ -2 & -5 & -5 & -2 \\ 2 & -2 & -1 & 3 \\ -1 & 0 & -5 & 0 \end{bmatrix}$$

Pooling

Pooling does a mathematical operation on the output of the convolution image. But you can also apply pooling directly to the image.



Building Blocks: Pooling

Pooling is a mathematical operation on the output of the convolution image. But you can also apply pooling directly to the image.

Pooling: Downsampling

Combine multiple adjacent nodes into a single node: max-pooling

$$\begin{bmatrix} 0 & -1 & -1 & 0 \\ -2 & -5 & -5 & -2 \\ 2 & -2 & -1 & 3 \\ -1 & 0 & -5 & 0 \end{bmatrix} \rightarrow \text{max} \rightarrow \begin{bmatrix} 0 \end{bmatrix} \quad (1)$$

$$\begin{bmatrix} 0 & 1 & -1 & -0 \\ -2 & -5 & -5 & -2 \\ 2 & -2 & -1 & 3 \\ -1 & 0 & -5 & 0 \end{bmatrix} \rightarrow \text{max} \rightarrow \begin{bmatrix} 0 & 0 \end{bmatrix} \quad (2)$$

- ⚡ Reduces the dimensionality of the input to subsequent layers and thus, the number of weights to be learned
- ⚡ Further, it protects the network from (slightly) noisy inputs

Downsampling with Stride

- ⚡ Only apply the convolution to some subset of the image
- ⚡ e.g., every other column and row = a **stride of 2**

$$\begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 2 & 2 & 1 & 0 \\ 0 & 2 & 4 & 4 & 2 & 0 \\ 0 & 1 & 3 & 3 & 1 & 0 \\ 0 & 1 & 2 & 3 & 1 & 0 \\ 0 & 0 & 1 & 1 & 0 & 0 \end{bmatrix} \times \begin{bmatrix} 0 & 1 \\ 1 & -2 \end{bmatrix} = \begin{bmatrix} -2 & \cdot & \cdot \\ \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot \end{bmatrix}$$

Downsampling with Stride

$$\begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 2 & 2 & 1 & 0 \\ 0 & 2 & 4 & 4 & 2 & 0 \\ 0 & 1 & 3 & 3 & 1 & 0 \\ 0 & 1 & 2 & 3 & 1 & 0 \\ 0 & 0 & 1 & 1 & 0 & 0 \end{bmatrix} \times \begin{bmatrix} 0 & 1 \\ 1 & -2 \end{bmatrix} = \begin{bmatrix} -2 & -2 & . \\ . & . & . \\ . & . & . \end{bmatrix}$$

Downsampling with Stride

$$\begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 2 & 2 & 1 & 0 \\ 0 & 2 & 4 & 4 & 2 & 0 \\ 0 & 1 & 3 & 3 & 1 & 0 \\ 0 & 1 & 2 & 3 & 1 & 0 \\ 0 & 0 & 1 & 1 & 0 & 0 \end{bmatrix} \times \begin{bmatrix} 0 & 1 \\ 1 & -2 \end{bmatrix} = \begin{bmatrix} -2 & -2 & 1 \\ \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot \end{bmatrix}$$

Downsampling with Stride

$$\begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 2 & 2 & 1 & 0 \\ 0 & 2 & 4 & 4 & 2 & 0 \\ 0 & 1 & 3 & 3 & 1 & 0 \\ 0 & 1 & 2 & 3 & 1 & 0 \\ 0 & 0 & 1 & 1 & 0 & 0 \end{bmatrix} \times \begin{bmatrix} 0 & 1 \\ 1 & -2 \end{bmatrix} = \begin{bmatrix} -2 & -2 & 1 \\ 0 & 1 & 1 \\ 1 & 2 & 0 \end{bmatrix}$$

Operation after Pooling

- ⚡ After applying convolution and pooling, we apply a non-linear activation, **ReLU**.
- ⚡ Then after that, you have more Convolution, more Pooling layers.
- ⚡ Finally, after several sequences of **Conv + Pool + ReLU**, we **flatten** the output and feed it to the fully connected layer (ANN).

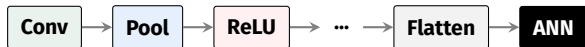
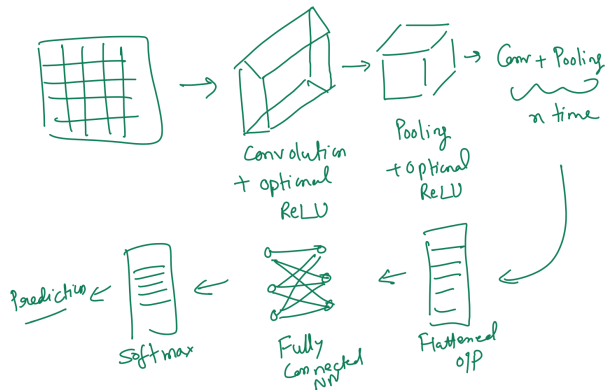
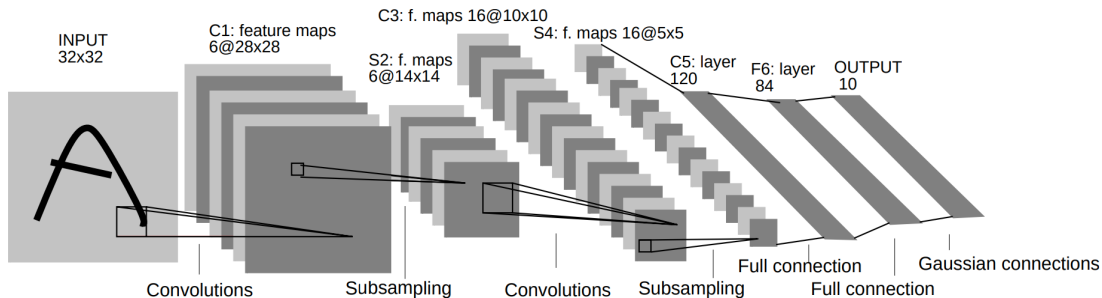


Image Classification

Then, we apply a softmax activation if our goal is to do supervised image classification.



Lenet



Source: http://vision.stanford.edu/cs598_spring07/papers/Lecun98.pdf

- ⚡ One of the earliest, most famous deep learning models – achieved remarkable performance at handwritten digit recognition ($< 1\%$ test error rate on MNIST dataset)
- ⚡ Used sigmoid (or logistic) activation functions between layers and mean-pooling, both of which are pretty uncommon in modern architectures

Tensor

We need a concept of higher-order matrices called a **Tensor**.

⚡ $X \in \mathbb{R}^{H \times W \times D}$ is an **order-3 tensor**. Consider it as D channels of matrices. (Note: $D = 1$ reduces it to a standard matrix).

Tensor Orders

- ⚡ **Order-0:** A Scalar value.
- ⚡ **Order-1:** A Vector.
- ⚡ **Order-2:** A Matrix. (*Corrected from your text*)
- ⚡ **Order-3:** A Cube (e.g., RGB Image).

We can also have **4 dimensions** if we consider opacity (e.g., RGBA in PNG).

Tensor to Vector

Given a tensor, we can arrange all numbers into a long vector following a **prespecified order**.

Example (Matrix Vectorization - Column First Order):

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} \xrightarrow{\text{MATLAB } A(:)} \begin{bmatrix} 1 \\ 3 \\ 2 \\ 4 \end{bmatrix}$$

Recursive Vectorization

To vectorize an **order-3 tensor**, we vectorize the first channel (matrix), then the second, and so on. This recursive process applies to tensors of any order > 3 .

Tensor in CNN

In CNNs, the convolution kernel will form an **order-4 tensor**.

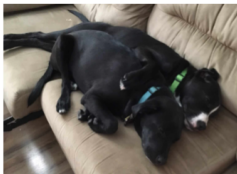
CNN Input Dimensions

- 1 **Depth:** Number of channels in a colored image.
- 2 **Filters:** Multiple filters learning specific features.
- 3 **Spatial Dimensions:** $H \times W$ of the image.

Tensor Orders (Examples):

- ⚡ **Order 3:** A **batch** of grayscale images.
- ⚡ **Order 4:** A **batch** of colored images or a single colored movie.
- ⚡ **Order 5:** A **batch** of movies.

Tensors



Example:
 $3 \times 4 \times 6$ tensor

4	1	2	16	3	6						
1		5	2	6	14	15	8				
5	26										
0	0										
	7										
		4	6	8	9	5	3				
		16	5	2	8	2	1				
		5	2	14	11	7	8				
		15	2	5	0	9	8				

Source: <https://www.cs.cmu.edu/~mgormley/courses/10601//slides/lecture17-cnn.pdf>

Further Reading

<https://www.cs.toronto.edu/~hinton/absps/NatureDeepReview.pdf>

http://vision.stanford.edu/cs598_spring07/papers/Lecun98.pdf

Modern Convolutional Neural Network

0	1	0	1	1	0	0	0	0	0	1	0	1	1	1	1	0	1	0	0
0	1	1	1	1	1	1	0	0	0	0	0	0	0	0	0	0	1	0	0
1	1	0	0	1	1	1	0	1	0	1	1	0	1	1	1	0	0	1	0

Modern Days of CNN Had Arduous Path

- 1 CNN didn't find its popularity easily.
- 2 There was a **lack of good datasets**.
- 3 **Kernel and ensemble methods** were still better until 2012.
- 4 Computer vision pipelines used methods such as *SIFT*, *SURF*, *bag of visual words*.
- 5 Researchers were using **hand-crafted features**.

The Paradigm Shift

However, **LeCun, Hinton, Ng**, and others thought features can be learnt too. **AlexNet** (after Alex Krizhevsky) proved that.

Finally Some Interesting and Useful Datasets

⚡ Large-Scale Benchmarks

- 1 **ImageNet (2009):** <https://huggingface.co/datasets/ILSVRC/imagenet-1k>
- 2 **CIFAR-10 & CIFAR-100:** <https://www.cs.toronto.edu/~kriz/cifar.html>
- 3 **LAION Dataset:** <https://laion.ai/projects/>

In addition, Yan LeCun used **MNIST**:

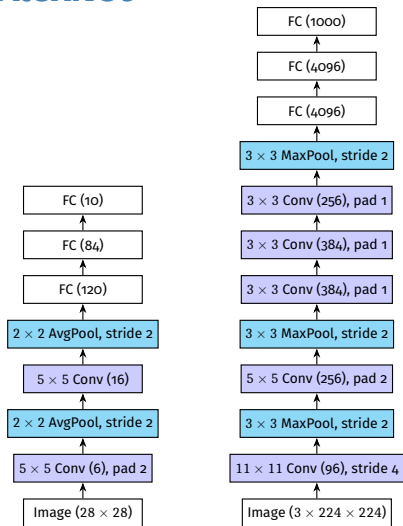
https://github.com/rahulbhadani/CPE490_590_Sp2025/tree/master/Data/MNIST

We also have **FashionMNIST**:

<https://github.com/zalandoresearch/fashion-mnist>

Note: These two are smaller size datasets.

AlexNet



From LeNet (left), AlexNet (right).

AlexNet Architecture

- 1 Convolution window shape: 11×11 , then 5×5 , and then 3×3 .
- 2 Network adds maxpooling of 3×3 with a stride of 2.
- 3 At the end, two huge fully connected layers with 4096 outputs.
- 4 ReLU activation function.

Block Thinking

Neurons



Layers



Blocks

Basic building blocks of CNN are sequence of **(i)** Convolutional layer with padding **(ii)** a nonlinearity such as a ReLU **(iii)** a pooling layer such as max-pooling to reduce the resolution.

But pooling layer reduces dimension by $\log_2 d$ for d dimension input, and it can happen fast exponentially.

- 1 **Simonyan and Zisserman (2014)** used multiple convolutional layer in between downsampling via max-pooling in the form of block.
- 2 **Two 3×3 convolution** have same effect as one 5×5 convolution layer: **deep and narrow networks significantly outperform their shallow counterparts.**

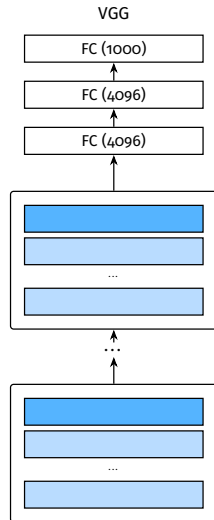
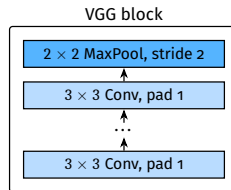
Three important papers in that area:

- 1 Simonyan, K. & Zisserman, A. "Very deep convolutional networks for large-scale image recognition." (2014).
- 2 Lavin, A. & Gray, S. "Fast algorithms for convolutional neural networks." (2016).
- 3 Liu, Z. et al. "A convnet for the 2020s." (2022).

VGG Block

VGG Block (**Simonyan and Zisserman (2014)**), consists of a sequence of convolutions with 3×3 kernels with padding of 1 (keeping height and width) followed by a 2×2 max-pooling layer with stride of 2 (halving height and width after each block).

- ⚡ VGG Network can be partitioned into two parts: the first consisting mostly of convolutional and pooling layers and the second consisting of fully connected layers that are identical to those in AlexNet.
- ⚡ But the convolutional layers are grouped in nonlinear transformations that leave the dimensionality unchanged, followed by a resolution-reduction step.



Network in Network (NiN)

Two main problems with AlexNet and VGGNet:

- 1 FCN at the end of the network has tremendous number of parameters.
- 2 Cannot add FCN earlier in the network otherwise it can destroy spatial information.

NiN:

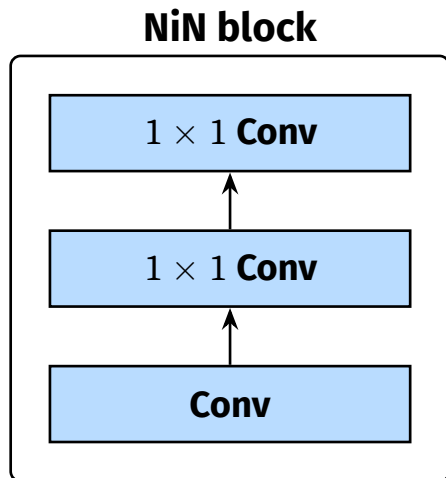
- 1 use 1×1 convolutions to add local nonlinearities across the channel activations
- 2 use global average pooling to integrate across all locations in the last representation layer

NiN Block

Input/Output of Convolutional layer is 4-d Tensor with no. of samples $\times$ channel $\times$ height $\times$ width.

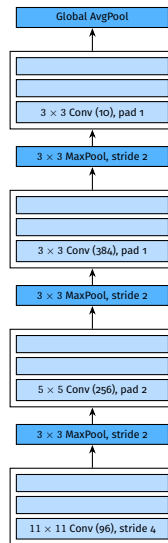
Input/output of FC layer are 2-D Tensor: no. of samples $\times$ features.

The idea behind NiN is to apply a fully connected layer at each pixel location (for each height and width). The resulting 1×1 convolution can be thought of as a fully connected layer acting independently on each pixel location.



NiN Model

- ⚡ **Initial Convolution Sizes:** NiN uses the same initial convolution sizes as AlexNet:
 - Kernel sizes are 11×11 , 5×5 and 3×3 respectively.
 - The numbers of output channels match those of AlexNet.
- ⚡ **Downsampling:** Each NiN block is followed by a max-pooling layer with a stride of 2 and a window shape of 3×3 .
- ⚡ **Architectural Shift:**
 - NiN avoids fully connected layers altogether.
 - Instead, NiN uses a NiN block with a number of output channels equal to the number of label classes.
 - Followed by a global average pooling layer, yielding a vector of logits.



GoogLeNet: Multi-Branch Networks

Szegedy, C. et al. (2015). *Going deeper with convolutions*. *Proceedings of the CVPR* (pp. 1–9).

Network Structure:

⚡ Stem for data ingestion:

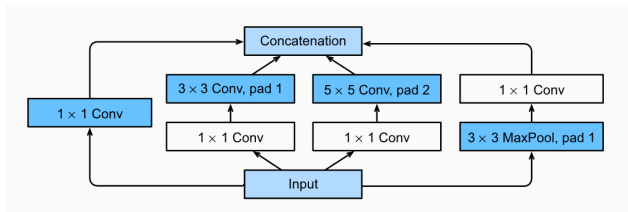
- First two or three convolutions.
- Operate directly on the image.

⚡ Body for data processing:

- Contains convolution blocks.
- It has concatenated multi-branch convolutions.

⚡ Head for prediction:

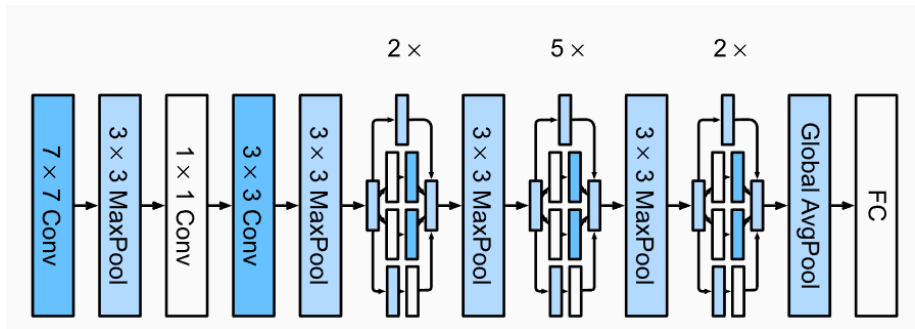
- Maps features to the required problem.
- Handles classification, segmentation, detection, or tracking.



Convolution block in GoogLeNet is the **Inception Block**.

GoogLeNet Model

GoogLeNet uses a stack of a total of 9 inception blocks, arranged into three groups with max-pooling in between, and global average pooling in its head to generate its estimates. Max-pooling between inception blocks reduces the dimensionality. At its stem, the first module is similar to AlexNet and LeNet.



Applying Batch Normalization in CNN

- ⚡ **Placement:** With convolutional layers, we can apply batch normalization after the convolution but before the nonlinear activation function.
- ⚡ **Per-Channel Logic:** The key difference from batch normalization in fully connected layers is that we apply the operation on a per-channel basis across all locations.
- ⚡ **Dimensions:**
 - Assume minibatches contain m training samples.
 - For each channel, output height is p and width is q .
 - Normalization is carried out over the $m \cdot p \cdot q$ elements per output channel simultaneously.
- ⚡ **Spatial Normalization:**
 - Collect values over all spatial locations to compute mean and variance.
 - Apply the same mean/variance within a given channel to every spatial location.
- ⚡ **Parameters:** Each channel has its own scale and shift parameters, both of which are scalars.

Other Advanced CNN Architecture

- 1 Xception - Deep Learning with Depthwise Separable Convolutions
<https://arxiv.org/abs/1610.02357>
- 2 DenseNet - Densely Connected Convolutional Neural Networks
<https://arxiv.org/abs/1608.06993v5>
- 3 MobileNetV1 - Efficient Convolutional Neural Networks for Mobile Vision Applications
<https://arxiv.org/abs/1704.04861v1>
- 4 MobileNetV2 - Inverted Residuals and Linear Bottlenecks
<https://arxiv.org/abs/1801.04381>
- 5 ConvNeXt - A ConvNet for the 2020s
<https://arxiv.org/abs/2201.03545>

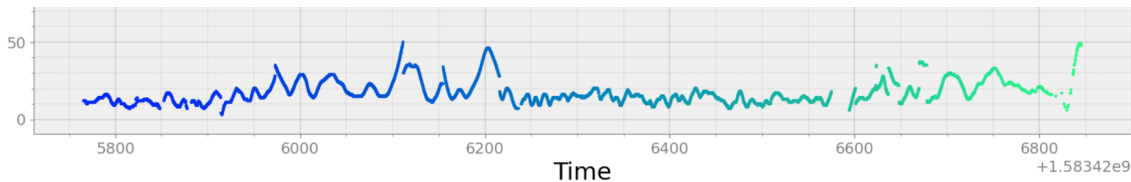
N-Gram Language Model

1	0	1	0	0	0	0	0	1	0	1	0	0	1	1	1	1	0	0	0
0	0	1	1	0	1	1	1	0	1	1	0	0	1	1	0	1	0	1	1
0	0	0	1	1	0	0	0	1	0	0	1	1	0	1	0	1	0	0	1

Motivation

We have seen that Convolutional Neural Networks are effective in capturing spatial relationships in data like images. Further, CNN is good for classification, and regression like tasks are complicated to implement with CNN.

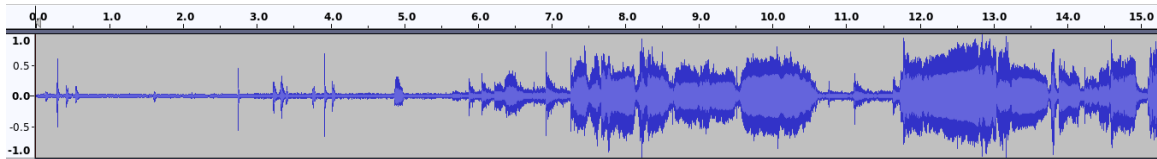
However, for data where sequential relationships are important (e.g., text, time series), we need something else.



Timeseries Data.

Some Example of Sequence Data

Speech Signal:



Text Sequences:

გამარჯობა! მე მიყვარს საქართველო - ჩვენი სილამაზით განთქმული ქვეყანა. საქართველო მდებარეობს კავკასიაში, შავი ზღვის აღმოსავლეთ სანაპიროზე. ჩვენი ქვეყანა ცნობილია

Language Model

Definition

A **language model** is a machine learning model designed to predict upcoming words in a sequence.

- 1 It assigns a **probability** to each possible next word, providing a *probability distribution*: $P(w_t \mid w_1, \dots, w_{t-1})$.
- 2 It can also assign a joint probability to an **entire sentence**: $P(w_1, \dots, w_n)$.

Which has a higher probability of appearing?

"all of a sudden I notice three guys standing on the sidewalk"

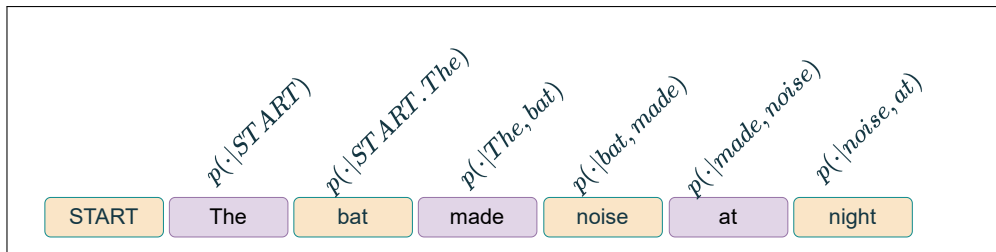
vs.

"on guys all I of notice sidewalk three a sudden standing the"

N-Gram Language Model

Core Objective

- ⚡ Simplest kind of Language Model.
- ⚡ **Goal:** Generate realistic-looking sentences in human language.
- ⚡ **Key Idea:** Condition on the last $n - 1$ words to sample the n -th word.



n-Gram Language Model

Question: How can we define a probability distribution over a sequence of length T?

The owl made noise at night
 w_1 w_2 w_3 w_4 w_5 w_6

n - Gram Model (n = 2)

Using chain rule:

$$p(w_1, w_2, \dots, w_T) = p(w_1)p(w_2|w_1)p(w_3|w_{1:2}) \cdots p(w_n|w_{1:n-1})$$

$$p(w_1, w_2, \dots, w_T) = \prod_{t=1}^T p(w_t|w_{t-1})$$

$$P(w_1, w_2, w_3, w_4, w_5, w_6) =$$

⚡ $p(w_1)$ (The)

⚡ $p(w_2|w_1)$ (The $\rightarrow$ owl)

⚡ $p(w_3|w_2)$ (owl $\rightarrow$ made)

⚡ $p(w_4|w_3)$ (made $\rightarrow$ noise)

⚡ $p(w_5|w_4)$ (noise $\rightarrow$ at)

⚡ $p(w_6|w_5)$ (at $\rightarrow$ night)

n-Gram Language Model

Question: How can we define a probability distribution over a sequence of length T?

The owl made noise at night
 w_1 w_2 w_3 w_4 w_5 w_6

n - Gram Model (n = 3)

$$p(w_1, w_2, \dots, w_T) = \prod_{t=1}^T p(w_t | w_{t-1}, w_{t-2})$$
$$P(w_1, w_2, w_3, w_4, w_5, w_6) =$$

⚡ $p(w_1)$ (The)

⚡ $p(w_2 | w_1)$ (The $\rightarrow$ owl)

⚡ $p(w_3 | w_2, w_1)$ (The, owl $\rightarrow$ made)

⚡ $p(w_4 | w_3, w_2)$ (owl, made $\rightarrow$ noise)

⚡ $p(w_5 | w_4, w_3)$ (made, noise $\rightarrow$ at)

⚡ $p(w_6 | w_5, w_4)$ (noise, at $\rightarrow$ night)

Learning an n-Gram Model

Question: How do we learn the probabilities for the n-Gram Model?

Answer: Calculate relative frequencies from training data:

$$p(w_t | w_{t-n+1}, \dots, w_{t-1}) = \frac{\text{count}(w_{t-n+1}, \dots, w_t)}{\text{count}(w_{t-n+1}, \dots, w_{t-1})}$$

Example: 3-Gram Probabilities

Training corpus with agricultural sentences:

- ⚡ ... cows eat grass ... (appears 2 times)
- ⚡ ... cows eat hay daily ... (appears 2 times)
- ⚡ ... cows eat corn ... (appears 3 times)

Calculation: $p(\text{corn} | \text{cows eat}) = \frac{3}{7} \approx 0.43$

3-Gram Counts (context: “cows eat”):

Next Word	Count
grass	2
hay	2
corn	3

Probability Distribution:

Next Word	Probability
grass	0.29
hay	0.29
corn	0.43

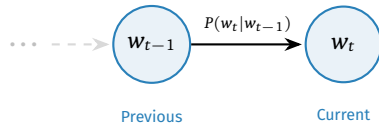
Markov Assumption

Definition

The assumption that the probability of a word depends only on the previous word is called a Markov assumption.

Markov Models

Markov models are the class of probabilistic models that assume we can predict the probability of some future unit without looking too far into the past.



Visualizing the first-order dependency.

How to estimate probabilities

Maximum Likelihood Estimation method:

Count words from a corpus, and normalize the counts so they fall between 0 and 1.
Computing 2-gram probability:

$$p(w_n|w_{n-1}) = \frac{C(w_{n-1}w_n)}{\sum_w C(w_{n-1}w)} = \frac{C(w_{n-1}w_n)}{C(w_{n-1})}$$

because the sum of all 2-gram counts that start with a given word w_{n-1} is equal to 1-gram count for that word w_{n-1} .

Example

```
<s> I am Sam </s>  
<s> Sam I am </s>  
<s> I do not like green peas and cheese</s>
```

$P(I|\text{<s>}) = 2/3 = 0.67$, $P(\text{<s>}|\text{Sam}) = 1/2 = 0.5$
 $P(\text{Sam}|\text{<s>}) = 1/3 = 0.33$, $P(\text{Sam}|\text{am}) = 1/2 = 0.5$
 $P(\text{am}|I) = 2/3 = 0.67$, $P(\text{do}|I) = 1/3 = 0.33$

Probability Estimation for n-gram parameter estimation

Relative Frequency Estimation:

⚡ **Definition:** n -gram probability is obtained by dividing the observed frequency of a particular sequence by the observed frequency.

⚡ **Terminology:** This approach is called **relative frequency**.

General Equation:

$$P(w_n | w_{n-N+1:n-1}) = \frac{C(w_{n-N+1:n-1} w_n)}{C(w_{n-N+1:n-1})}$$

Estimating the probability of a word given its $N - 1$ previous words.

Question

How to get probability

$P(\langle s \rangle \text{ I want English food } \langle /s \rangle) = ?$

using 2-gram model?

Dealing with Large n-gram Models

The Multiplication Problem

What would happen if you multiply enough n-grams together?

Dealing with Large n-gram Models

The Multiplication Problem

What would happen if you multiply enough n-grams together?

→ How would you deal with it?

Dealing with Large n-gram Models

The Multiplication Problem

What would happen if you multiply enough n-grams together?

→ How would you deal with it? **We can instead do log probabilities.**

Dealing with Large n-gram Models

The Multiplication Problem

What would happen if you multiply enough n-grams together?

→ How would you deal with it? **We can instead do log probabilities.**

Context Expansion

How do we deal with longer context, like 3-gram, 4-gram?

Dealing with Large n-gram Models

The Multiplication Problem

What would happen if you multiply¹ enough n-grams together?

→ How would you deal with it? **We can instead do log probabilities.**

Context Expansion

How do we deal with longer context, like 3-gram, 4-gram?

For 3-gram, we can use two pseudo initial words e.g.

$$P(I \mid \langle s \rangle \langle s \rangle)$$

Dealing with Large n-gram Models

The Multiplication Problem

What would happen if you multiply¹ enough n-grams together?

- How would you deal with it? **We can instead do log probabilities.**

Context Expansion

How do we deal with longer context, like 3-gram, 4-gram?

For 3-gram, we can use two pseudo initial words e.g.

$$P(I \mid \langle s \rangle \langle s \rangle)$$

Efficient Computation Points:

- **Quantize probabilities** using only 4-8 bits (instead of 8-byte floats).
- Use **sorted arrays and merge sorts** to efficiently build the probability tables in a minimal number of passes through a large corpus.

Evaluating LMs

Extrinsic Evaluation

Embed into an application and evaluate how much the application improves.

It is comparative: compare two candidate LMs on tasks such as speed recognizer and machine translation.

Intrinsic Evaluation

Measuring quality of model independent of application.

Example: perplexity.

Perplexity of N-gram Model: The perplexity of an LM on a test set is the inverse probability of the test set normalized by the number of words or tokens.

We normalize by taking the N-th root. If we have test set $W = w_1 w_2, \dots w_N$:

$$\begin{aligned} \text{perplexity}(W) &= \sqrt[N]{\frac{1}{P(w_1 w_2 \dots w_N)}} \\ &= \sqrt[N]{\prod_{i=1}^N \frac{1}{P(w_i | w_1 \dots w_{i-1})}} \end{aligned}$$

Using chain rule of probabilities

⚠ Caveat: An intrinsic improvement in perplexity doesn't guarantee an extrinsic improvement in performance.

Perplexity: Weighted Average Branching Factor

- ✓ Lower perplexity means the model is more confident and accurate.

Branching factor of a language: The number of possible words that can follow any word.

- ⚡ **Deterministic:** Only 1 word follows; factor is 1.
- ⚡ **Uniform:** V words equally likely; factor is V .
- ⚡ **General:** Average number of possible next words, weighted by their probabilities.

Uniform Case

If $P(w_i) = \frac{1}{V}$, then:

$$\text{Perplexity} = (V^N)^{\frac{1}{N}} = V$$

Example: Bigram Model

Test seq: $I \rightarrow \text{love} \rightarrow \text{NLP}$ ($N = 3$)

$P(I|\langle s \rangle) = 1.0$, $P(\text{love}|I) = 0.5$, $P(\text{NLP}|\text{love}) = 0.8$

Step 1: Local Branching Factors

$$B_1 = 1/1.0 = 1$$

$$B_2 = 1/0.5 = 2$$

$$B_3 = 1/0.8 = 1.25$$

Step 2: Perplexity (Weighted Avg)

$$\begin{aligned} \text{PPL} &= \sqrt[3]{B_1 \times B_2 \times B_3} \\ &= \sqrt[3]{1 \times 2 \times 1.25} = \sqrt[3]{2.5} \approx \mathbf{1.357} \end{aligned}$$

Interpretation:

Avg meaningful next-word choices = 1.357 (not naive vocab size)

Sampling from a Language Model

Process:

- 1 Start with initial context (n-1 words)
- 2 Roll weighted die for next word or choose a random number between 0 and 1.
- 3 Slide window and repeat

Sampling Demonstration:

Step	Context	Sampled Word
1	[START] The	bat
2	The bat	made
3	bat made	noise
4	made noise	at
5	noise at	night
6	at night	[END]

Generated Sentence:

The bat made noise at night

Sampling introduces diversity but preserves local coherence through n-gram constraints.

N-gram Language Models Don't Work Well

- ⚡ **Data Sparsity:** As n increases, the number of possible n -grams grows exponentially, but they might never appear in a training corpus.
- ⚡ **The Zero-Probability Problem:** Unseen n -grams receive zero probability, which is problematic.
- ⚡ **Context Limitations:** Long-range dependencies are impossible, or computationally intractable to capture.
- ⚡ **Lack of Semantic Generalization:** N -grams treat words as discrete symbols and fail to generalize.

Recurrent Neural Network

0	1	1	0	0	1	0	0	0	1	0	0	0	0	1	0	1	1	0	0
1	1	1	1	0	1	0	0	1	1	1	1	1	0	0	0	1	1	0	1
0	0	1	1	1	0	0	0	0	0	0	1	0	1	1	1	1	1	1	1

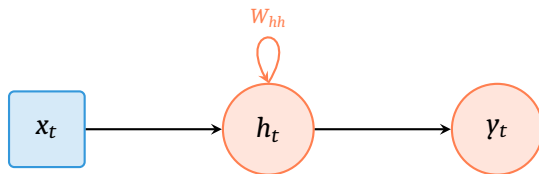
Recurrent Connections and Recurrent Neural Networks

Recurrent Connection

Introduction of a cycle in a network.

Recurrent Neural Network (RNN)

Any neural network that contains a cycle within its network connection.



Hidden State

Folded representation of a cycle in an RNN.

There are many types of RNN. We will see the **Elman network** (1990, Elman).

Elman Network

The equations for the Elman network are:

$$h_t = g(Uh_{t-1} + Wx_t)$$
$$y_t = f(Vh_t)$$

where g is an activation function (e.g., tanh or sigmoid).

At the end, output layer also contains softmax:

$$y_t = \text{softmax}(Vh_t)$$

🎯 The goal is to learn the weights:

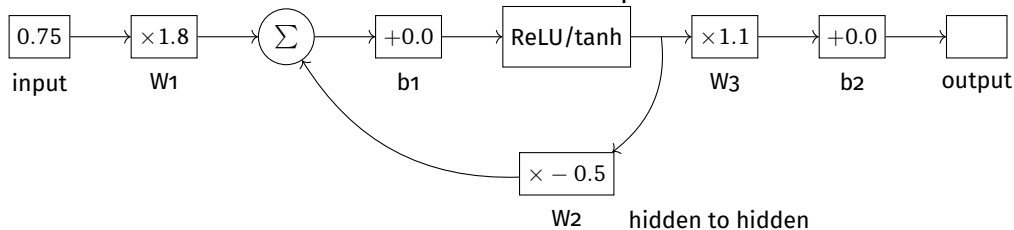
- U (Recurrent)
- W (Input)
- V (Output)

📘 At the end output layer, softmax is often used.

See an Example RNN Network

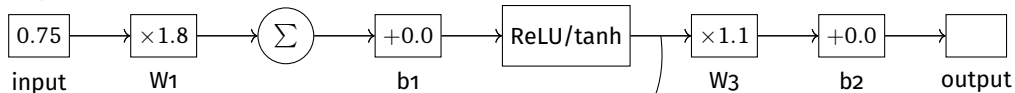
The feedback loop in Recurrent Neural Networks (RNNs) networks makes it possible to establish long-range dependence on sequential data.

To understand how these networks process sequential data over time, we can "unroll" the network and use numeric timeseries as an example.



Unrolling the RNN Network

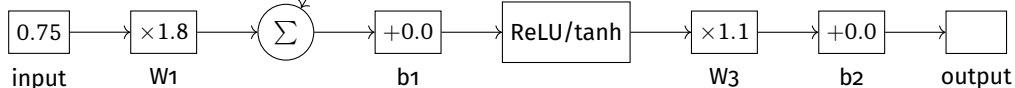
Yesterday's value



Predicted Today's value

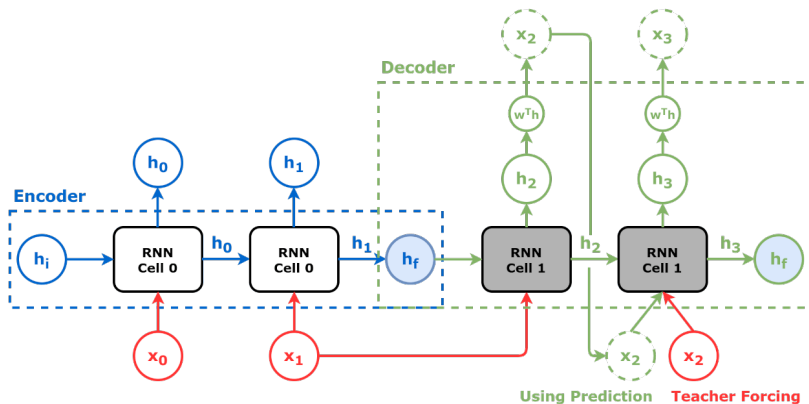
$\times -0.5$
 W_2 hidden to hidden

Today's value



Predicted Tomorrow's value

RNN Cell enforces the recurrent connection.



Source: <https://github.com/dvgodoy/dl-visuals>

Teacher Forcing

The idea that the predicted value is not used as input but the true value is used is called **teacher forcing**.

Vanishing Gradient / Exploding Gradient Problem

As we see that when we unroll the network, we have multiplication of W_2 (recurrent weight) with itself many times.

Exploding Gradient

If $W_2 > 1$, then eventually, the gradient will become really large ($W_2^{50} \gg 1$).

Vanishing Gradient

If $W_2 < 1$, then eventually, the gradient will become really small ($W_2^{50} \ll 1$).

Vanishing Gradient / Exploding Gradient Problem

Consider an RNN with ReLU activation:

$$\begin{aligned}h_t &= \max(0, W_1 x_t + W_2 h_{t-1} + b_1) \\ y_t &= W_3 h_t + b_2\end{aligned}\tag{3}$$

The backpropagation through time (BPTT) is used for training sequence data. The gradient of the loss (L) with respect to W_2 is:

$$\frac{\partial L}{\partial W_2} = \sum_{k=1}^t \frac{\partial L}{\partial y_k} \cdot \frac{\partial y_k}{\partial h_k} \cdot \frac{\partial h_k}{\partial W_2}$$

where k is the index for time.

Vanishing Gradient / Exploding Gradient Problem

$$\frac{\partial h_k}{\partial W_2} = \frac{\partial h_k}{\partial h_{k-1}} \cdot \frac{\partial h_{k-1}}{\partial W_2} = W_2 \cdot \frac{\partial h_{k-1}}{\partial W_2} = W_2 \cdot \left(W_2 \cdot \frac{\partial h_{k-2}}{\partial W_2} \right) = W_2^2 \cdot \frac{\partial h_{k-2}}{\partial W_2}$$

Continuing this pattern, we get:

$$\frac{\partial h_k}{\partial W_2} = W_2^{k-1} \cdot \frac{\partial h_1}{\partial W_2}$$

This shows that the gradient $\frac{\partial h_k}{\partial W_2}$ is proportional to W_2 and the previous gradients.

As you see, if $|W_2| > 1$, then the gradient grows exponentially (exploding gradient).

If $|W_2| < 1$, then the gradient vanishes.

Long-term Dependency Problem in RNN

RNN also suffers from **long-term dependency problem**. When the gap between the relevant information and the point where it is needed becomes very large, prediction becomes poorer.

Short Gap (Success)

Consider an example: *"The clouds are in the sky"*.

Next word in the sequence *"The clouds are in the"*, is definitely **"sky"**, no further context is needed.

Long Gap (Problem)

But if you consider *"I speak fluent French"*, then the next word in the sequence *"I speak fluent"*, may or may not be French.

We need additional context. In such a case, **RNN is not a useful model**.



Long Short-Term Memory (LSTM)

0	0	1	0	1	0	1	1	0	0	0	1	1	0	1	1	0	1	0	0
1	1	0	0	1	0	0	0	0	0	0	0	1	0	1	1	1	0	1	0
0	1	1	0	0	1	0	0	0	0	0	1	0	0	0	0	1	1	1	0

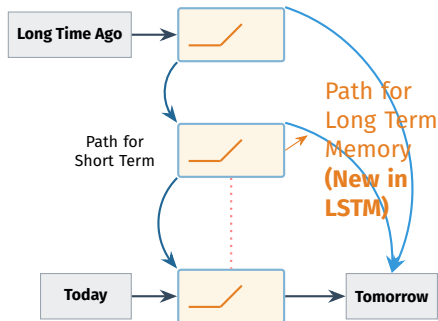
Long Short-Term Memory (LSTM)

In LSTM, we modify connections so that there are **two separate paths** for long-term and short-term dependencies.

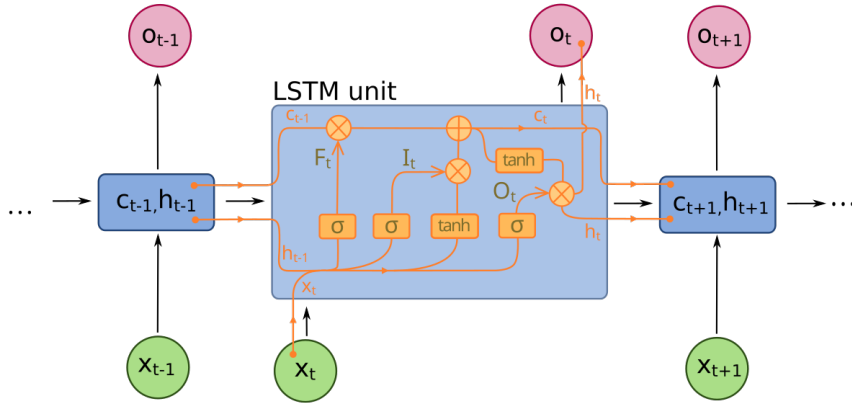
⚡ LSTM specifically uses the **tanh** and **sigmoid** activation functions:

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$



LSTM



Source: <https://github.com/dvgodoy/dl-visuals>

What's special in LSTM

Central Idea

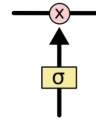
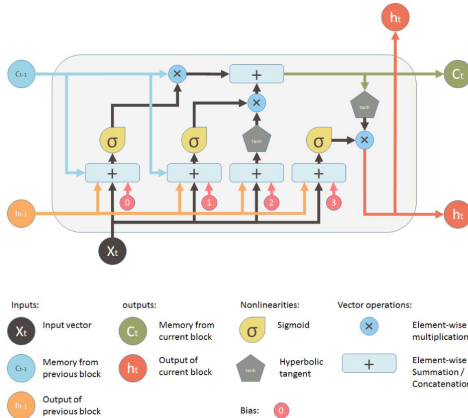
LSTM Introduces cell state and gating mechanisms to preserve long-term information. A memory cell (interchangeably block) which can maintain its state over time, consisting of an explicit memory (aka the cell state vector) and gating units which regulate the information flow into and out of the memory.

There are 3 kinds of gating mechanism:

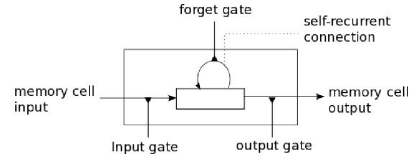
- ⚡ **Forget Gate:** Decides what information to discard
- ⚡ **Input Gate:** Decides what new information to store
- ⚡ **Output Gate:** Decides what to output

LSTM Memory Cell

LSTM Memory Cell



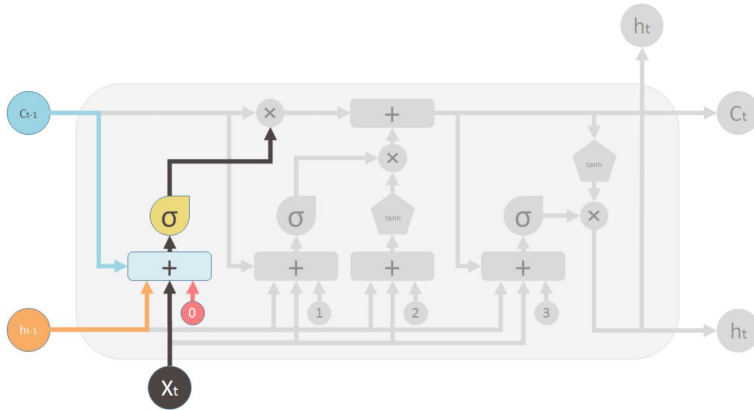
Gate (sigmoid layer followed by pointwise multiplication)



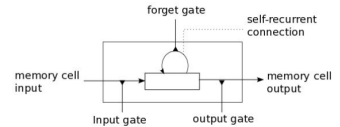
Simplified schematic for reference

Source: <https://pages.cs.wisc.edu/~shavlik/cs638/lectureNotes/Long%20Short-Term%20Memory%20Networks.pdf>

Forget Gate

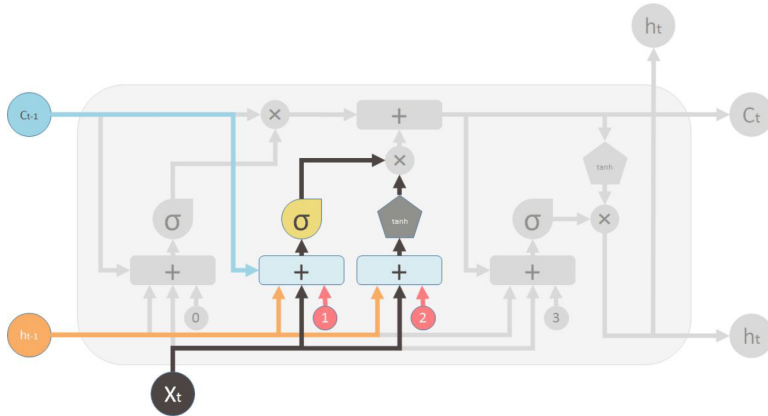


$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$

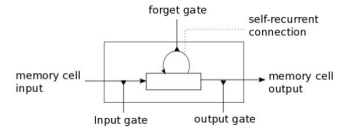


Source: <https://pages.cs.wisc.edu/~shavlik/cs638/lectureNotes/Long%20Short-Term%20Memory%20Networks.pdf>

Input Gate



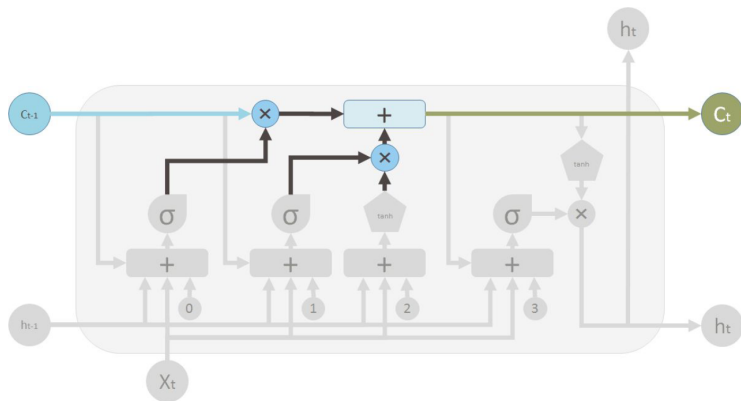
$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$
$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$$



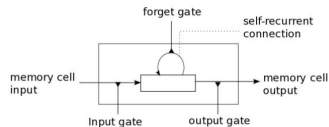
Source: <https://pages.cs.wisc.edu/~shavlik/cs638/lectureNotes/Long%20Short-Term%20Memory%20Networks.pdf>

Memory Update

The cell state vector aggregates the two components (old memory via the forget gate and new memory via the input gate)

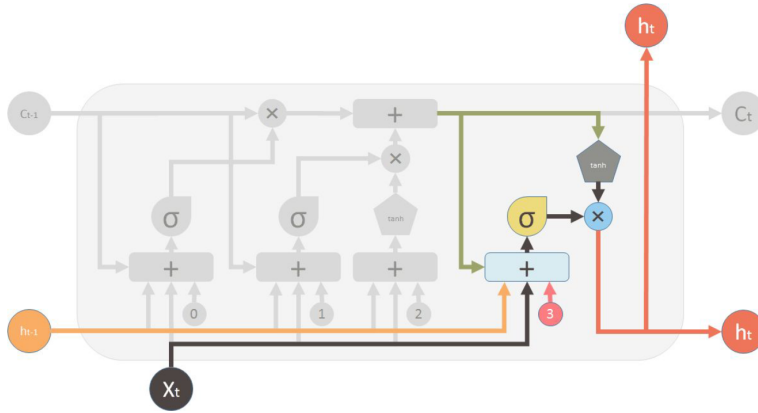


$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$



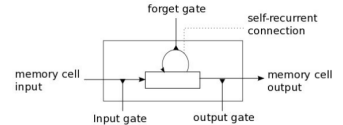
Source: <https://pages.cs.wisc.edu/~shavlik/cs638/lectureNotes/Long%20Short-Term%20Memory%20Networks.pdf>

Output Gate



$$o_t = \sigma(W_o [h_{t-1}, x_t] + b_o)$$

$$h_t = o_t * \tanh(C_t)$$



Source: <https://pages.cs.wisc.edu/~shavlik/cs638/lectureNotes/Long%20Short-Term%20Memory%20Networks.pdf>

Gating Mechanism in LSTM

1. Forget Gate (f_t)

Output of f_t tells the percentage the previous cell state will be kept in calculating the updated cell state.

⚡ **0** = forget everything

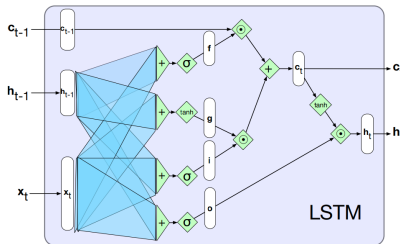
⚡ **1** = keep everything

2. Input Gate (i_t)

Output if i_t tell the percentage of the new candidate that will be used in calculating the updated cell state.

⚡ **0** = ignore new info

⚡ **1** = accept fully

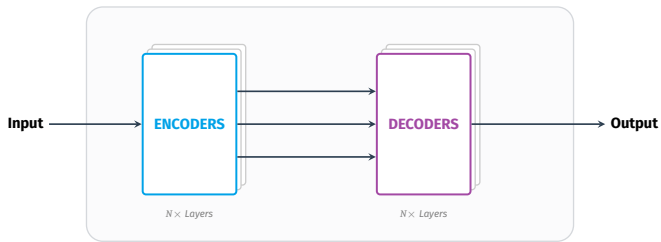


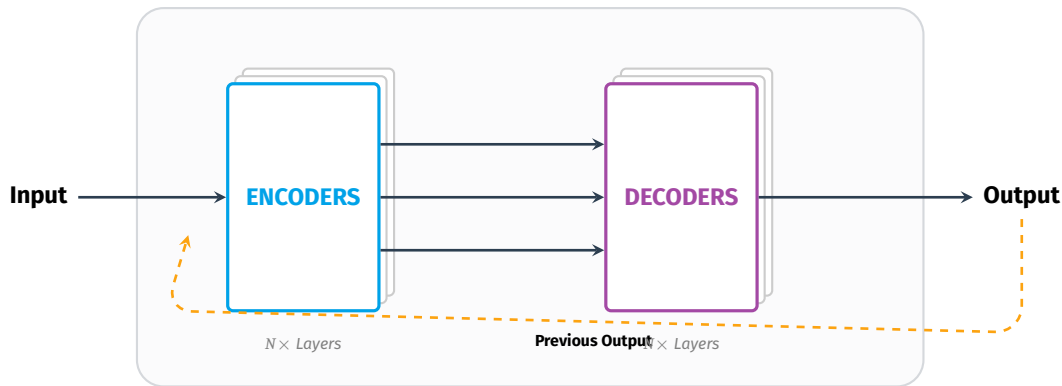
Transformer

0	1	1	0	0	1	1	1	0	0	0	0	0	1	1	0	1	1	0	0
1	1	1	1	0	1	0	1	1	0	0	1	1	0	1	1	1	1	1	0
1	1	1	0	1	0	0	1	0	1	1	0	0	1	0	1	1	1	0	1

It Started with Attention!

- 🤖 **Behind modern genAI chatbots.**
- ⌨️ **Generative AI text-based tools**
- 📄 First proposed in 2017 in the paper **Attention is All You Need**



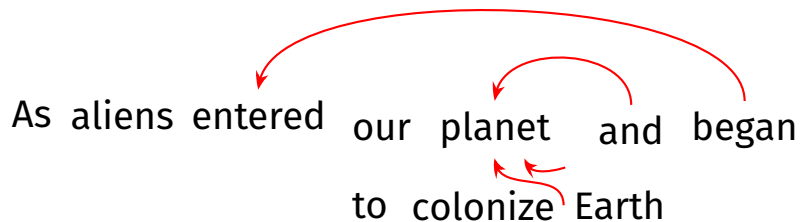


This is called auto-regressive transformer architecture.

Addressing Long Range Context

Consider the text:

As aliens entered our planet and began
to colonize Earth



The diagram shows the sentence "As aliens entered our planet and began to colonize Earth" with red arrows indicating dependencies. A long arrow connects "entered" to "began", showing a long-range dependency. Another arrow connects "planet" to "began". Two shorter arrows connect "colonize" to "Earth".

Transformer has longer Context Window

As the model generates a text word-by-word, it has the ability to reference or tend to words that are relevant to the generated word.



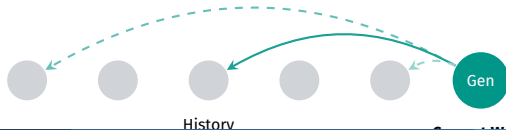
How the model learns which word to tend is done via back propagation.



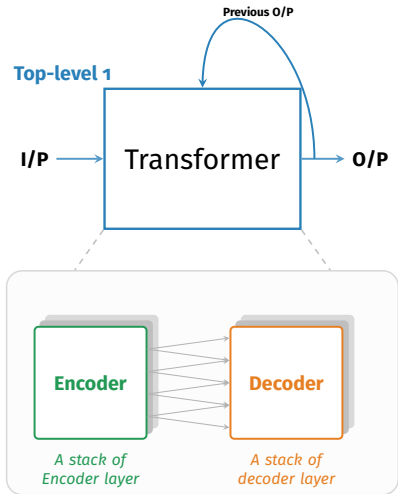
A transformer has an infinite history to look at, only limited by the available memory.

Check the RAM prices in 2026!!!

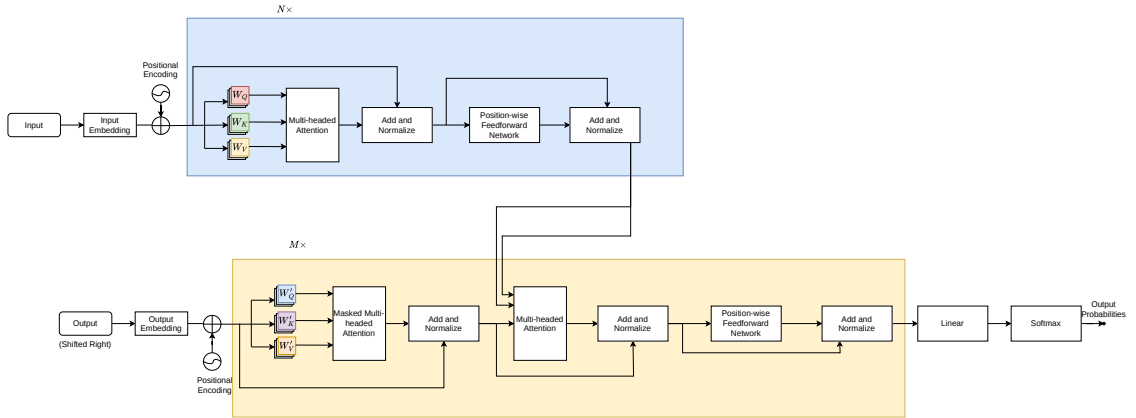
Yet, transformers have fixed context window due to the cost associated with the inference.



Transformer Architecture



Unraveling Encoder-Decoder Architecture



Step 1: Tokenization

Tokenization: A text is converted to different tokens, which are a part of a fixed vocabulary.

"A quick brown fox jumped over the lazy dog"

Word-level

{ A, quick, brown, fox, jumped, over, the, lazy, dog }

Subword tokens

jumps over → {jump, s, over}

Character-level

Dog → { D/d, o, g }

Then we create numerical mapping: "The cat sat on the mat" → { The:1, cat:2, sat:3, on:4, the:5, mat:6 }

Padding: In order to keep all the input to the same length, we add padding at the end of the sequence.

"The cat sat on the mat [PAD] [PAD] [PAD]"

Tokenization: Byte-pair Encoding

❗ Due to fixed context window of a transformer, an efficient tokenization is necessary.

Byte-pair Encoding

- ➔ Start with all individual characters as tokens, (a, b, c, etc.)
- ➔ Look through your training data and find the most common pair of adjacent characters
- ➔ Merge the most frequent pair of adjacent tokens into a new token and add to your vocabulary
- ➔ Repeat steps 2-3 thousands of times until you have your desired vocabulary size

Byte-pair Encoding Example: Step 1

Sample Corpus:

{low, low, low, new, new, wide, wide, wide, wide}

Step 1: Split words into individual characters

(Add `</w>` = end-of-word marker to track word boundaries)

⚡ low → l, o, w, `</w>` x3

⚡ new → n, e, w, `</w>` x2

⚡ wide → w, i, d, e, `</w>` x4

Initial Vocabulary

l, o, w, n, e, i, d, `</w>`

Byte-pair Encoding Example: Step 2

Merge (Most Frequent Pair)

w + </w>	(3+2 = 5)
e + </w>	(4)
l + o	(3)
o + w	(3)
n + e	(2)
e + w	(2)
w + i	(4)
i + d	(4)
d + e	(4)

⊕ **Most frequent pair:** w + </w>

⚙️ **Merge into new token:** w</w>

↻ **Updated word splits:**

⚡ low → l, o, w</w>

⚡ new → n, e, w</w>

⚡ wide → w, i, d, e, </w>

Updated Vocabulary: l, o, w, n, e, i, d, </w>, w</w>

Byte-pair Encoding Example: Step 3

Second Merge (Iterative Process)

Recount pairs (after first merge)

➡ **New most frequent pair:** e + </w> (4)

⚙️ **Merge into new token:** e </w>

↻ **Updated word splits:**

⚡ low → l, o, w</w>

⚡ new → n, e, w</w>

⚡ wide → w, i, d, e</w>

Updated Vocabulary:

l, o, w, n, e, i, d, w</w>, e</w>

↻ Repeat this merge process until you reach your target vocabulary size!

Byte-pair Encoding Example: Step 4

▶▶ Next merge: $w+i \rightarrow wi$ (4)

▶▶ Then merge: $d+e</w> \rightarrow de</w>$ (4)

- ⚡ Common words become full tokens
- ⚡ Less common words stay split
- ⚡ Unknown words later split into existing subwords

WordPiece Tokenization Scheme

Instead of the absolute count as the criteria for merging, it calculates the merge score as:

WordPiece Merge Score

$$\text{score} = \frac{\text{count}(\text{pair})}{\text{count}(\text{token1}) \times \text{count}(\text{token2})}$$

➡ Here, the algorithm merges the pair with the highest score.

Prioritizes pairs that appear together more often than by chance.

Step 2: Embedding Layer

 **Embedding layer** is there to enable a model to learn about relationship between words, tokens, or other inputs.

✂ It is a matrix with dimensions: (**vocabsize** $\times$ **dmodel**)

 For example, we can have:

⚡ **vocabsize** = 6

⚡ **dmodel** = 3

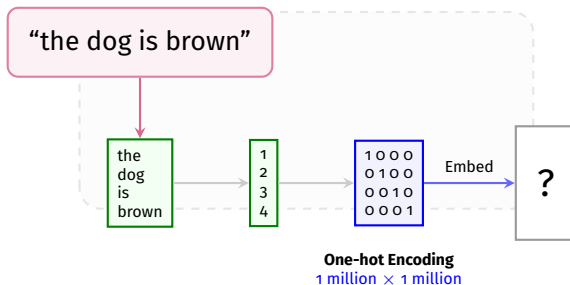
If we pass **dog** to the embedding layer, the layer would return $[0.12, -0.5, 0.88]$.

Would be initialized with random numbers in the beginning and then will be learnt via backpropagation eventually.

Token	v1	v2	v3
dog	0.12	-0.5	0.88
cat	-0.1	0.33	0.12
mat	0.45	0.11	-0.9
fox	0.01	0.92	0.05
jump	-0.8	-0.2	0.44
quick	0.3	0.06	0.29

Example 6: 6×3 Matrix 

We also perform one hot encoding



But $1 \text{ million} \times 1 \text{ million}$ matrix seems a bit too much. Right?

Embedding Layer

Embedding layer can be used to map one-hot encoding matrix to a lower dimensional representation.



Each word can be represented as a `dmodel`-dimensional vector.



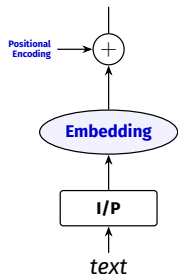
Initially, you can use `rand` function to randomly initialize the embedding matrix.

$$\underbrace{\begin{bmatrix} \cdot & \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot & \cdot \end{bmatrix}}_{\text{One Hot Encoding} \atop \text{vocabsize} \times \text{vocabsize}} \times \underbrace{\begin{bmatrix} \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot \end{bmatrix}}_{\text{Embedding Matrix} \atop \text{vocabsize} \times \text{dmodel}} = \underbrace{\begin{bmatrix} \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot \end{bmatrix}}_{\text{Embedding for the sequence} \atop \text{vocabsize} \times \text{dmodel}}$$

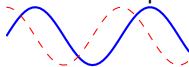
Note: Matrix multiplication acts as a "lookup" for specific row vectors.

The embedding matrix will be updated and optimized using backpropagation.

Step 3: Positional Encoding



- ⚡ As transformer itself doesn't provide information about positions, we have to encode them separately.
- ⚡ **Positional Encoding** is used to provide a relative position for each token or word in a sequence. When reading a sentence, each word is dependent on the word around it. Some words have different meaning in different context.
- ⚡ To encode the position, we use cosine, and sine.



Consider L as the maximum number of positions or vectors in the embedding.

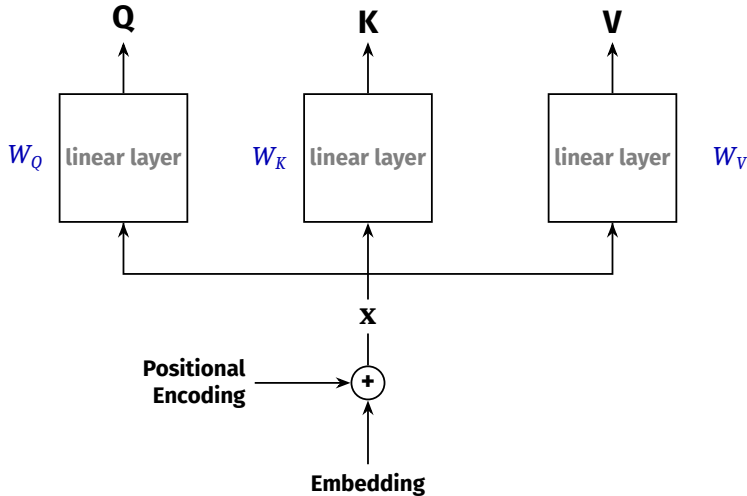
```
for each k = 0 to L-1
  for each i = 0 to dmodel/2
    PE(k, 2i) = sin( k / n^{2i/dmodel})
    PE(k, 2i+1) = cos( k / n^{2i/dmodel})
```

$$PE = \begin{bmatrix} \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot \end{bmatrix}$$

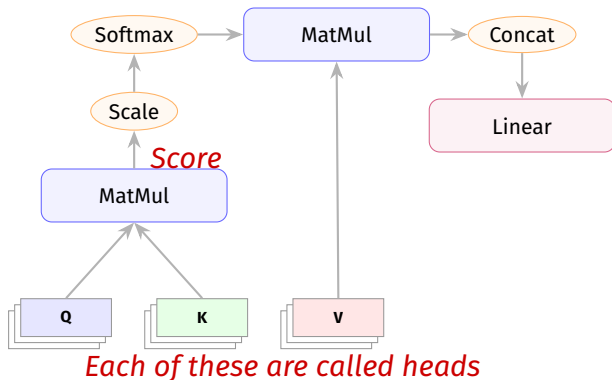
n is user-defined, set as 10000 by the original author.

Once PE Matrix is calculated, it is added to the embedding matrix calculated in the previous step.

Step 4: Multihead Attention



Self-Attention in Multi-Head Attention



Self-attention allows a model to associate each individual word in the input to the other words in the input.

Example: In “How are you”, the model learns to associate *How* strongly with *you*.

⚡ **Q:** Query, has dimension d_k

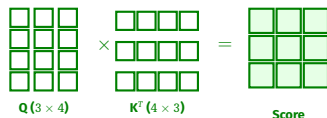
⚡ **K:** Key, has dimension d_k

⚡ **V:** Value, has dimension d_v

The Q, K, V Mechanism

Query, Key, Value notion comes from the retrieval system. In this system, a query is mapped using a key and it returns some values.

Score Calculation



Similarity Mapping

	Hi	How	are	you
Hi	98	27	10	12
How	27	89	31	67
are	10	31	91	54
you	12	67	54	92

Score matrix tells us how much focus one word should put on another words

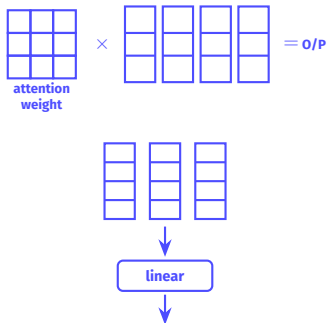
Scaling Step:

$$\text{Attention} = \text{Softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

Where d_k is the dimension of keys/queries.

Attention Weights

$$\text{Softmax}(\text{scaledscore}) = \text{attentionweight}$$



Since we can have many Q, K, V, each of them are called heads. That's why we have the name **multi-head attention**.

- ⚡ Each output as a result of each *head* gets concatenated to get a single vector before passing through the linear layer.
- ⚡ Each head learns something different.

Multi-headed attention output is added to the original input. This is called **residual connection**.

Multi-head Attention

Linearly project the queries, keys, and values h times with different learned projections to d_k , d_k , and d_v dimensions, respectively. Attention is performed in parallel, yielding d_v -dimensional output values, then concatenated and projected again.



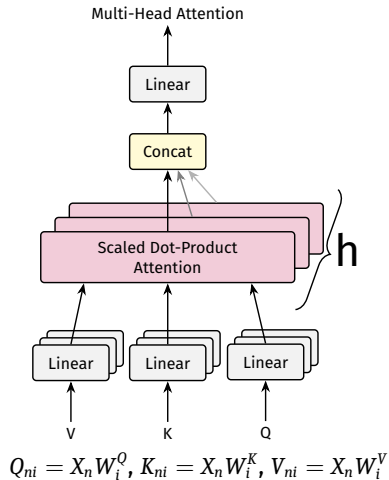
$$\text{Multihead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O$$



$$\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$

$$\text{Lightning bolt icon } W_i^Q \in \mathbb{R}^{d_{\text{model}} \times d_k}, W_i^K \in \mathbb{R}^{d_{\text{model}} \times d_k}, W_i^V \in \mathbb{R}^{d_{\text{model}} \times d_v}, \\ W^O \in \mathbb{R}^{hd_v \times d_{\text{model}}}$$

Original paper sets $d_k = d_v = d_{\text{model}}/h = 64$.



Position-wise Feedforward Network

$$FFN(x) = \max(0, xW_1 + b_1)W_2 + b_2$$

Layer Normalization

→ Input to Layer Norm is a single vector of dimensionality d_{model} and the output is that vector normalized, of the same dimensionality.

- 1 Calculate the mean μ and the standard deviation σ over the elements of the vector to be normalized.

If $\mathbf{x}$ is the embedding, then

$$\mu = \frac{1}{d_{model}} \sum_{i=1}^{d_{model}} x_i$$
$$\sigma = \sqrt{\frac{1}{d_{model}} \sum_{i=1}^{d_{model}} (x_i - \mu)^2}$$

- 2 Then the normalized output is

$$\hat{\mathbf{x}} = \frac{\mathbf{x} - \mu}{\sigma}$$

In the implementation, we have two learnable parameters γ and β such that

$$\text{LayerNorm}(\mathbf{x}) = \gamma \frac{\mathbf{x} - \mu}{\sigma} + \beta$$

Step 5: Decoder

	<start>	How	are	you
<start>		X	X	X
How		✓	X	X
are		✓	✓	X
you		✓	✓	✓

Shift Logic:

I/P Side → <start> How are you

O/P Side → *shifted* <start> <start> How are you

Architecture

Decoder layer will have two multiheaded attention. The first attention layer is similar to what we saw in the encoder side.

- ⚡ But the second attention layer takes input from the output of the encoder side.
- ⚡ Then, after passing through the normalization layer, we add output from two attention layers and then normalize again.

This process matches the encoder input to the decoder input, allowing decoder to decide which encoder input is relevant to put focus on.

Language Model Head

The language model head is the last component of the transformer.

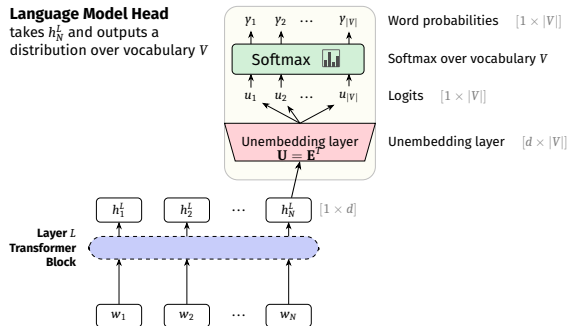
The job of the language model is to take the output of the final transformer layer from the last token N and use it to predict the upcoming word at position $N + 1$.

Linear Layer There is a linear layer that takes output h_N^L which is the output token embedding at position N from the final block L of shape $1 \times d_{model}$ to the logit vector (score vector) and then have a single score for each of the $|V|$ possible words in the vocabulary V . The logit vector has dimensionality $1 \times |V|$.

Unembedding Layer Unembedding layer E^T where E is the embedding matrix of shape $|V| \times d_{model}$, with same weight is used to map back from the embedding shape $1 \times d_{model}$ to $1 \times |V|$.

Softmax Turns the logits into probabilities.

Language Model Head takes h_N^L and outputs a distribution over vocabulary V



We might sample a word with highest probability or some other sampling strategy.

Three Architectures of Language Models

Encoder-decoder (Original Transformer Architecture)

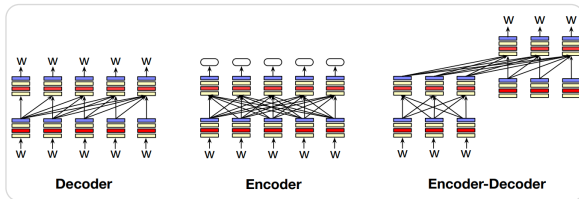
takes a series of tokens as input and output a series of tokens.

Encoder

takes a sequence of tokens as input and output their vector representation. (BERT, RoBERTA, etc.)

Decoder (The most commonly used ones now)

takes a series of tokens as input and iteratively generates an output token one at a time. (GPT, Claude, Lllama, Mistral)



References and Readings

Must Read

- 1 N-gram Language Models: <https://web.stanford.edu/~jurafsky/slp3/3.pdf>
- 2 Large Language Models: <https://web.stanford.edu/~jurafsky/slp3/7.pdf>
- 3 Transformers: <https://web.stanford.edu/~jurafsky/slp3/8.pdf>
- 4 RNNs and LSTMs: <https://web.stanford.edu/~jurafsky/slp3/13.pdf>
- 5 <https://pages.cs.wisc.edu/~shavlik/cs638/lectureNotes/Long%20Short-Term%20Memory%20Networks.pdf>
- 6 Attention is All You Need: <https://arxiv.org/abs/1706.03762>
- 7 Scaling Laws for Neural Language Models: <https://arxiv.org/abs/2001.08361>
- 8 DeepSeek-V3 Technical Report: <https://arxiv.org/abs/2412.19437>
- 9 The Llama 3 Herd of Models: <https://arxiv.org/abs/2407.21783>

Recommended Reading

- 1 Large Language Models: A Survey: <https://arxiv.org/abs/2402.06196>
- 2 Neural Machine Translation by Jointly Learning to Align and Translate: <https://arxiv.org/abs/1409.0473>